Company Handover: Using and Extending the IFC Dashboard

1. Purpose of the Dashboard

This dashboard has been developed as a reusable internal tool for exploring BIM-related data across multiple building projects. It allows users to view schedules, inspect individual building elements (such as doors), review associated visual media, and access 3D geometry via an integrated web-based viewer.

The system is designed so that new projects can be added, updated, or removed without modifying the dashboard code, using only a structured folder arrangement.

2. System Overview

The solution consists of two main components:

- **Streamlit Dashboard**
  Provides the user interface for schedules, images, metadata, and project navigation.
- **Web-Based 3D Viewer (xeokit)**
  Runs as a separate local service and is linked to from the dashboard for geometric inspection.

The dashboard and viewer are deliberately separated to improve performance, reliability, and maintainability.

3. Project Folder Structure

All project-specific content is managed through folders. The only locations that need to be updated for new projects are:

```
data/
images/
```

All other files (application code, viewer files, converters) can remain unchanged.

4. Adding a New Project (Step-by-Step)

Step 1: Create a Project Data Folder

Create a new folder under data/ using the project name:

```
data/new_project/
```

Step 2: Add Schedule Data

Place any schedule CSV files inside the project folder:

```
data/new_project/
├── door_schedule.csv
├── room_schedule.csv
```

Each CSV file will automatically:

- Appear as a new tab in the dashboard
- Be displayed as an interactive table

No configuration is required.

Step 4: Add Project Images

Architectural visuals can be added to:

```
images/new_project/
├── floor_plan.png
├── render.png
├── elevation.png
```

These images are displayed in the Building Visuals section of the dashboard.

Step 5: Add Excel-Derived Graphs (Optional)

If analytical outputs are produced in Excel, export them as images and place them in:

images/new_project/excel_graphs/

Supported formats include .png, .jpg, and .jpeg.
These will be shown automatically under Excel-Based Analysis.

6. Removing or Updating a Project

To remove a project:

1. Delete the corresponding folder from data/
2. Remove associated folders from images/
3. (Optional) Remove geometry files from the viewer directory

The dashboard will automatically update to reflect the available projects.

3d Viewer - User Guide Quick Start

For End Users (Viewing Only)

1. Open your web browser (Chrome or Edge recommended)
2. Navigate to the dashboard URL provided by your administrator
3. Select a building from the dropdown menu in the sidebar
4. Navigate the 3D model using:

- **Left click + drag** = Rotate view
- **Right click + drag** = Pan around
- **Scroll wheel** = Zoom in/out

5. View door schedules and inspection data in the main tabs

For Administrators (Adding New Buildings)

Follow the instructions in Sections below.

Step 6: Register Your Project in the Dashboard

Edit `app.py` and add your project to the `projects` dictionary:

```python
projects = {
    "Office Block": "office",
    "Apartment Building": "apartment",
    "Town Hall": "town_hall",
    "Your New Project": "your_project_name"  # ← ADD THIS
}
```

Setting Up Inspection Data

JSON File Format

For each element (door as example), create a JSON file with the same name as its folder. This json file is linked to show an image and a link to a youtube video

```json
{
    "id": "D101",
    "type": "Internal Door",
    "level": "Ground Floor",
    "fire_rating": "FD30",
    "width_mm": 900,
    "height_mm": 2100,
    "description": "Main corridor access door leading to office wing",
    "image": "D101.png",
    "video": {
        "type": "youtube",
        "src": "https://www.youtube.com/embed/VIDEO_ID_HERE"
    }
```

}

Image Requirements

- Format: PNG or JPG
- Recommended size: 800x600 pixels
- Filename must match the image field in JSON
- Place in the same folder as the JSON file

YouTube Videos

To add a walkthrough video:

1. Upload video to YouTube
2. Get the embed URL (looks like: https://www.youtube.com/embed/VIDEO_ID)
3. Add to JSON as shown above

Converting IFC Files for Web Viewing

Prerequisites

- Node.js installed
- xeokit-convert tool downloaded (see installation guide)

Basic Conversion Command (through prompt window)

```bash
cd C:\ifc_dashboard\xeokit-convert
node convert2xkt.js -s "C:\path\to\your\building.ifc" -o "C:\ifc_dashboard\xeokit-simple-viewer-main\buildings\your_building.xkt" --maxIndicesForEdge 50000 -l
```

For Large IFC Files (over 200MB)

```bash
node --max-old-space-size=8192 convert2xkt.js -s "C:\path\to\large\building.ifc" -o "C:\ifc_dashboard\xeokit-simple-viewer-main\buildings\your_building.xkt" --maxIndicesForEdge 200000 -l
```

Expected Conversion Times

IFC Size	Estimated Time
Under 100MB	2-5 minutes
100-300MB	10-15 minutes
300-600MB	20-30 minutes

Common Warnings (Safe to Ignore)

- `No basis found for brep!` - Complex curved surfaces, does not affect model
- `Geometry has too many triangles` - Increase `--maxIndicesForEdge` value

Running the Dashboard Locally - What You Need

- Python 3.10+ installed
- Streamlit installed (`pip install streamlit`)
- The dashboard files in `C:\ifc_dashboard`

Step 1: Start the HTTP Server

Open **Terminal 1** (Anaconda Prompt):

```
cd C:\ifc_dashboard
python -m http.server 8000
```

Keep this terminal running. You will see:

```text
Serving HTTP on :: port 8000 ...
```

Step 2: Start the Streamlit Dashboard

Open **Terminal 2** (Anaconda Prompt):

```bash
cd C:\ifc_dashboard
streamlit run app.py
```

Keep this terminal running. Streamlit will open your browser automatically.

Step 3: Access the Dashboard

- Dashboard URL: http://localhost:8501
- 3D Viewer URL (if accessed directly): http://localhost:8000/dashboard.html

Why Two Terminals?

Terminal	Port	Purpose
Python HTTP Server	8000	Serves XKT files (bypasses browser security)
Streamlit	8501	Runs the dashboard interface

Both must remain running for the dashboard to work during initial viewing before dashboard deployment.

Stopping the Dashboard

Press Ctrl + C in each terminal window and close command prompt.

Deploying for Use

Cloud Deployment (Recommended)

Once deployed, users only need a web browser - no local servers required.

Option A: Streamlit Cloud (Free)

1. Push your code to a GitHub repository
2. Go to share.streamlit.io
3. Connect your GitHub account
4. Select the repository
5. Click "Deploy"

Option B: Render (Free Tier)

1. Push your code to GitHub
2. Create a new Web Service on Render
3. Connect your repository
4. Set start command: streamlit run app.py
5. Click "Create Web Service"

Making Changes After Deployment

How Updates Work

Once deployed, you can update the dashboard at any time:

Action	Process
Fix a typo	Edit file → Save → Push to GitHub
Add new building	Create folder → Convert IFC → Update code → Push
Change styling	Edit CSS in `dashboard.html` → Push
Add new feature	Edit `app.py` → Test locally → Push

No Downtime

- Cloud platforms (Streamlit Cloud, Render) use **zero-downtime deployment**
- Users don't see errors during updates
- Old version runs until new version is ready

Rollback (Undo Changes)

If an update causes problems:

Via GitHub:

```
git revert HEAD
git push
```

Via Streamlit Cloud:

1. Go to app dashboard
2. Click "Revert to previous version"
3. Select working version

Hosting XKT Files for Deployment

Since XKT files can be large (30MB+), host them separately:

Platform	Free Tier	Best for
AWS S3	5GB	Large enterprise
Cloudflare R2	10GB	Fast global access

Platform	Free Tier	Best for
Google Drive	15GB	Simple sharing
Dropbox	2GB	Small teams

After Deployment

Share the URL for viewing

Troubleshooting

"3D viewer is blank"

Check	Solution
Is HTTP server running?	Run `python -m http.server 8000`
Does XKT file exist?	Check `xeokit-simple-viewer-main/buildings/`
Browser console errors?	Press F12, look for red text

"No schedules found"

Check	Solution
CSV in correct folder?	Must be in `data/your_project/`
File extension?	Must be `.csv` (not `.CSV`)
Folder name matches?	Must match project name in `projects` dictionary

"Door image not available"

Check	Solution
Image file exists?	Check door folder
Filename matches JSON?	`"image": "D101.png"` must match actual filename
File format supported?	Use PNG or JPG

"HTTP Server not running" message

Start the HTTP server in a separate terminal:

```bash
cd C:\ifc_dashboard
python -m http.server 8000
```

Conversion fails with "Out of memory"

Add memory parameter:

```bash
node --max-old-space-size=8192 convert2xkt.js ...
```

Clicking on 3D model does nothing

This is a known limitation with very large models (over 500,000 triangles). Use the tabular schedules to access element data instead.

Folder Structure Reference

Complete Required Structure

```text
C:\ifc_dashboard\
|
 ├── app.py                ← Main Streamlit dashboard
 ├── dashboard.html            ← 3D IFC viewer
|
 ├── data\                 ← All project data
|    ├── office\           ← Office project
```

```
│   │   ├── door_schedule.csv
│   │   ├── window_schedule.csv
│   │   └── doors\
│   │       ├── D101\
│   │       │   ├── D101.json
│   │       │   └── D101.png
│   │       └── D102\
│   │           ├── D102.json
│   │           └── D102.png
│   │
│   ├── apartment\              ← Apartment project
│   │   └── (same structure)
│   │
│   └── town_hall\              ← Town Hall project
│       └── (same structure)
│
├── images\                     ← Building visuals
│   ├── office\
│   │   ├── floor_plan.png
│   │   ├── render.png
│   │   ├── elevation.png
│   │   └── excel_graphs\
│   │       └── chart1.png
│   ├── apartment\
│   │   └── (same structure)
│   └── town_hall\
│       └── (same structure)
│
└── xeokit-simple-viewer-main\
    └── buildings\              ← Converted XKT files
        ├── office.xkt
        ├── apartment.xkt
        └── town_hall.xkt
```

Naming Rules

Element	Rule	Example
Project folder	Lowercase, no spaces	town_hall

Element	Rule	Example
CSV files	Any name, ends with `.csv`	`door_schedule.csv`
Door folders	Any name, matches JSON	`D101`
JSON files	Same as folder name	`D101.json`
Image files	Any name, matches JSON	`D101.png`
XKT files	Same as project name	`town_hall.xkt`

Support

For technical issues or questions about customization, refer to:

- **xeokit SDK documentation:** https://xeokit.io/docs
- **Streamlit documentation:** https://docs.streamlit.io
- **IFC conversion guide:** https://github.com/xeokit/xeokit-convert

Version History

Version	Date	Changes
1.0	April 2026	Initial release - Office, Apartment, Town Hall projects

Company Handover

3D Model Integration and Geometry Management (xeokit)

1.Geometry File Location and Naming Rules

All converted 3D models are stored in a single directory:

```
xeokit-simple-viewer-main/
└── buildings/
    ├── office.xkt
    ├── apartment.xkt
    └── town_hall.xkt
```

Naming requirements:

- Each .xkt file must be named exactly the same as the project identifier used in the dashboard.
- Filenames must be lowercase and contain no spaces.

If the naming rule is followed, the dashboard will automatically link to the correct model.

2. Converting IFC Files to XKT Format

Building geometry must be converted from IFC into the XKT format before it can be viewed in the browser. This is handled using the xeokit-convert tool.

Standard Conversion (Most Files)

Shell
```
node convert2xkt.js -s "path\to\building.ifc" -o "path\to\buildings\project_name.xkt" -l
```
Show more lines

Large IFC Files

For files larger than approximately 200 MB, additional memory should be allocated:

Shell
```
node --max-old-space-size=8192 convert2xkt.js -s "path\to\large.ifc" -o "project_name.xkt" --
maxIndicesForEdge 200000 -l
```
Show more lines

The conversion process may display warnings for complex geometry. These are expected and do not usually indicate failure.

3. Adding a New Building Model

To add a new building to the system:

1. Convert the IFC file to XKT format.
2. Place the resulting .xkt file in:
3. xeokit-simple-viewer-main/buildings/
4. Ensure the filename matches the project name used in the dashboard.
5. (If required) Add the project to the dashboard's project list.

No further configuration is required.

4. Running the Viewer Locally

The xeokit viewer is served using a lightweight local HTTP server.

Ports Used:

Service	Port	Function

HTTP Server	8000	Serves 3D viewer and XKT files
Streamlit	8501	Dashboard interface

5. Updating or Removing Models

To update an existing building:

- Replace the existing .xkt file with a newly converted version using the same filename.

To remove a building from the viewer:

- Delete the corresponding .xkt file from the buildings directory.
- Remove its associated data and images if no longer required.